



Security Assessment



Item 29 Internal Report

May 2026

Prepared for ether.fi

Disclaimer

This document is intended for **internal** use only

Draft

Initial Commit Hash	Final Commit Hash
7b5c2767e	2349e53e

Low Severity Issues

L-01 executeRecovery can be front-run to prevent safes from rescuing funds		
Severity: Low	Impact: Medium	Likelihood: Low
Files: TopUpV2.sol	Status: Acknowledged	

Description: The new `executeRecovery` function is a bridged call that is meant to allow safes to self-serve themselves and recover unsupported assets stuck in the `TopUp` contract. However `TopUpFactory` still implements the `processTopUp()` flow which can be invoked by anyone – this function takes any array of tokens, and for each that has a non-zero balance it transfers them to the Factory for bridging.

`FactoryTopUp` only bridges supported tokens, however `processTopUp()` transfers any token, regardless if it is supported or not.

This allows for anyone to snipe an inflight `executeRecovery()` bridge transaction and call `processTopUp()` right before that, which would move the funds into the Factory and revert the recovery. This causes the EthFi team to get involved and use the `recoverFunds()` flow in the Factory

Recommendations: Consider making `processTopUp()` & `processTopUpFromContracts()` permissioned or alternatively only process the enabled tokens, which would make the self-serve recovery path the only possible flow.

Customer's response: Acknowledged – *"Worst case is griefing: processTopUp sweeps the unsupported token to the Factory, executeRecovery reverts with NoBalanceToRecover, and the LZ packet stays retryable. Funds are not lost — the Factory's existing admin recoverFunds() path remains available as the fallback (the same path we used pre-recovery-module)."*

Not worth restricting processTopUp for a griefing-only scenario."

Fix Review: Acknowledged

Informational Issues

I-01. Check deployment salt when sending a bridged call to prevent wasting fees

Description: Currently the salt required to produce the expected deterministic safe address is [only validated at destination chain](#) in `AssetRecoveryDispatcher`. However this is not efficient as the invalid bridged call has already been sent and the fee paid.

JavaScript

```
if (TOPUP_FACTORY.getDeterministicAddress(p.salt) !== p.safe) revert SaltDoesNotMatchSafe();
```

It would be better to also add another salt validation at the top of the `recover()` function in `AssetRecoveryModule` to ensure that initially invalid messages will not be sent and waste unnecessary ETH on gas and fees.

Recommendation: Add salt validation at the sending stage of the bridged call

Customer's response: Acknowledged – *"LZ fees here are paid by ether.fi, not the user, and the salt is already validated twice before the message goes out:"*

1. In our backend before we sponsor the call.
2. Authoritatively on the destination chain in `AssetRecoveryDispatcher`.

A third check in `recover()` would be redundant."

Fix Review: Acknowledged

I-02. Implementation theft risk

Description: The deployment scripts use the `_deployCreate3` function, which is going to skip deployment when the contract has already been deployed:

```
function _deployCreate3(bytes memory creationCode, bytes32 salt) internal
returns (address deployed) {
    deployed = _predictImpl(salt);
    if (deployed.code.length > 0) return deployed; // idempotent
```

Since Nick's factory is used, anyone may front-run the deployment and steal the implementation addresses for the following contracts:

- [AssetRecoveryModule](#)
- [AssetRecoveryDispatcher](#)
- [TopUpV2](#)

Recommendation:

- Verify the bytecode of the `AssetRecoveryModule` before adding it as a whitelisted module in the data provider
- Verify the bytecode of the `AssetRecoveryDispatcher` before setting in the `TopUpV2`
- Verify the bytecode of the `TopUpV2` implementation before upgrading the beacon.

Customer's response: Acknowledged – "

Mitigated operationally. The deployment runbook (scripts/recovery/README.md) and the 3CP-secure PRs that wire these contracts both require bytecode verification before the operating safe:

- *whitelists the module,*
- *sets the dispatcher on TopUpV2, or*
- *upgrades the beacon.*

VerifyRecoveryDeployment.s.sol asserts deployed addresses match predictions post-deploy."

Fix Review: Acknowledged

I-03. AssetRecoveryModule not compatible with EIP-712

Description: EtherFiSafe and the rest of the safe core already sign with EIP-712 typed data. However, the AssetRecoveryModule will directly create a compute the digest, without using a `_domainSeparator` and adhering to the EIP-712.

Recommendation: For consistency, consider using eip-712 for the AssetRecoveryDispatcher.

Customer's response: Acknowledged – *"Consistency-only. The current digest binds all relevant fields (nonce, safe, token, recipient, salt, destEid, keccak256(lzOptions)) and signers are recovered against the Safe's owner set. Not changing the signing scheme for an in-flight module."*

Fix Review: Acknowledged

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.